AI-Based Platform to Support the Employment of Astana IT University Graduates

First Submission of Materials, Probationary Period 1

By

Kozhabek Ayadil Nurlanuly
Group SE-2425

Amantay Zhan Adiluly
Group SE-2419



Astana IT University

Supervisor: Orazova Arailym Zhanbolatkyzy
Senior Lecturer

October 2026
Astana

Submission Information

Field	Details
Project	AI-Based Platform to Support the Employment of Astana IT University Graduates
Team	Kozhabek Ayadil Nurlanuly, SE-2425; Amantay Zhan Adiluly, SE-2419
Supervisor	Orazova Arailym Zhanbolatkyzy, Senior Lecturer
Repository	https://github.com/vbl0-0/AITU-Career
Submitted version	https://github.com/vbl0-0/AITU-Career/tree/prob-period-1
Task tracker	https://github.com/users/vbl0-0/projects/1
Prototype	https://vbl0-0.github.io/AITU-Career/ — screen walkthrough; full application runs locally
Diagram sources	https://github.com/vbl0-0/AITU-Career/tree/prob-period-1/docs/pre-defense-1/diagrams
Report date	24 October 2026
Earlier prepared version	https://github.com/vbl0-0/AITU-Career/tree/prob-period-1-before-academic-20261024

Completed work

- Analysis: proposal and scope, a targeted review of 17 sources including 8 scientific publications, comparison of four systems against eight common criteria, and five technology decisions.
- Requirements: 12 functional and 8 non-functional requirements with priorities, acceptance criteria and verification methods.
- Design: system context, protected use cases, component architecture, data relationships and an API contract; readable exports and editable diagram sources.
- Working version: FastAPI and PostgreSQL backend; React interface; authentication and roles; graduate portfolio, vacancies, explained skill matching, recommendations and administration.
- Prototype: actual application captures and a public screen walkthrough; Kazakh, Russian and English interfaces in the locally running application.
- Verification: 60 backend tests and 20 browser tests passed; translation parity, production build and Alembic consistency were checked. The detailed results are in Chapter 9.
- Project management: a shared repository and GitHub Project with assigned tasks, original deadlines, current issue states and links to actual commits and pull requests.
- Supervisor feedback: the messages of 24 October 2026 and the reporting changes made in response are recorded in Chapter 11.

Known limitations

- The vacancy seed contains 30 fictional demonstration positions. The HeadHunter adapter is implemented, but live provider access and a real vacancy import have not been verified.
- The verified matching baseline uses deterministic catalogue and evidence rules. The optional embedding adapter has not been evaluated with a real model.
- Independent relevance labels and ranking-quality experiments are not yet available; passing functional tests establishes software behavior rather than matching effectiveness.
- The public prototype presents captured screens. Authentication, editing and live calculations require the locally running backend.
- Docker runtime, a comprehensive accessibility audit and independent human peer review remain outstanding.

- User interviews were optional for this submission and were not conducted. Their proposed role in later validation is described in Chapter 2.

Outstanding tasks

- Prepare the frozen evaluation dataset and independent relevance labels (GitHub issue #10).
- Compare the B0/B1/B2 matching methods and record ranking quality, latency and error analysis (issue #11).
- Introduce reciprocal human review of substantive pull requests and retain review comments, follow-up fixes and actual contribution records.
- Complete the final file/link check, submit through the form and retain the real confirmation (issue #18).

Chapters and repository files

The report is generated from versioned project content and evidence. The following paths identify its exported chapters and the principal implementation or evidence sources.

Chapter	Repository files
1. Project Proposal	docs/pre-defense-1/01-section.html; docs/pre-defense-1/submission.json
2. Evidence of Relevance	docs/pre-defense-1/02-section.html; docs/pre-defense-1/sources.json
3. Literature Review	docs/pre-defense-1/03-section.html; docs/pre-defense-1/sources.json
4. Existing System Comparison	docs/pre-defense-1/04-section.html; scripts/submission_content_en.py
5. Technology Comparison	docs/pre-defense-1/05-section.html; scripts/submission_content_en.py
6. Requirements Specification	docs/pre-defense-1/06-section.html; backend/app/ai/matching.py; backend/tests/test_matching.py
7. Architecture and Diagrams	docs/pre-defense-1/07-section.html; docs/pre-defense-1/diagrams; docs/pre-defense-1/evidence/openapi.json
8. UX and Prototype	docs/pre-defense-1/08-section.html; frontend/src/pages; docs/pre-defense-1/prototype
9. Verification and Evaluation Plan	docs/pre-defense-1/09-section.html; docs/pre-defense-1/evidence/validation.json; backend/tests; frontend/tests
10. Project Management and Risks	docs/pre-defense-1/10-section.html; docs/TEAM_DELIVERY.md; docs/pre-defense-1/evidence/development-history.json
11. Supervisor Feedback	docs/pre-defense-1/11-section.html; docs/pre-defense-1/evidence/supervisor-feedback-2026-10-24.json

Table of Contents

Submission Information ..... **i**

1. Project Proposal **1**

1.1 Problem 1

1.2 Target users 1

1.3 Aim and objectives 1

1.4 Scope 1

1.5 Team 1

2. Evidence of Relevance **2**

2.1 Sources of evidence 2

2.2 Findings and assumptions 2

2.3 Further validation 2

3. Literature Review **2**

3.1 Search scope 2

3.2 Literature comparison 3

3.3 Synthesis and research questions 3

4. Existing System Comparison **4**

4.1 Systems and criteria 4

4.2 Comparison 4

4.3 Implications for AITU Career 4

5. Technology Comparison **4**

5.1 Technology alternatives 4

5.2 Architectural decisions 5

6. Requirements Specification **5**

6.1 Priorities and traceability 5

6.2 Functional requirements 5

6.3 Non-functional requirements 6

6.4 Matching and readiness rules 7

6.5 AI functionality and implementation status 7

6.6 Worked example 7

7. Architecture and Diagrams **8**

7.1 System context 8

7.2 Use cases 9

7.3 Components 9

7.4 Data model and analysis flow 10

7.5 API and editable sources 10

8. UX and Prototype **11**

8.1 Prototype access 11
8.2 User scenarios 11
8.3 UX states and accessibility 11
8.4 Screen examples 12
9. Verification and Evaluation Plan 16
9.1 Verification results 16
9.2 Dataset and matching baselines 17
9.3 Independent labels and evaluation metrics 18
9.4 Execution responsibilities and planned artifacts 18
9.5 Validity threats 19
10. Project Management and Risks 19
10.1 Team contributions 19
10.2 Task allocation, deadlines and progress 19
10.3 Recorded development history 20
10.4 Git workflow and reciprocal peer review 20
10.5 Risks 21
11. Supervisor Feedback 21
11.1 Reviews received 21
11.2 Comments and decisions 22
11.3 Follow-up 22
Appendix A. References 22
Appendix B. Submission Checklist and Evidence Status 24

1. Project Proposal

1.1 Problem

Problem statement. Graduates need to compare their educational and practical experience with heterogeneous IT job requirements and identify an actionable next step. The existence and scale of this need specifically among AITU graduates remain hypotheses until local user discussions are completed. Research on skill extraction supports the technical relevance of the task, but does not establish local demand [S01–S05].

1.2 Target users

Target users are senior AITU students and graduates, plus an administrator of the educational prototype. A career-center representative is a potential requirements expert. An employer dashboard is outside the current MVP.

1.3 Aim and objectives

Aim. Develop and evaluate a reproducible prototype that explains how a graduate profile covers IT vacancy skill requirements, identifies skill gaps and suggests development actions. The matching percentage is not a probability of employment.

Table 1.1. Aim and objectives

Objective	Verifiable outcome
Investigate the domain	At least 12 meaningfully used sources including eight scientific publications; comparison of four systems. Optional user research is planned for later validation.
Specify requirements	A priority, acceptance criterion and verification method for each mandatory requirement.
Design the system	System context, use cases, components, data relationships and API responsibilities.
Implement the core journey	Profile → skills → vacancy → explained match → development actions.
Evaluate the approach	Compare exact matching, normalization and optional semantics under one fixed protocol.
Prepare submission evidence	Unified report, editable diagrams, traceable tasks and actual verification evidence.

1.4 Scope

Included: authentication and graduate/admin roles; AITU-oriented profiles and education; skills with proficiency levels; projects, certificates and experience; vacancies and saved jobs; transparent weighted matching; saved analyses with staleness detection; recommendations and administration. The local seed contains 30 fictional vacancies clearly identified as demonstration data.

Excluded from this stage: automated hiring decisions, validated prediction of employment, university SSO/DU/Moodle integration, large-scale collection of external vacancies, production SLA, employer accounts and training a large language model. Visual similarity to university services does not imply integration or institutional endorsement.

1.5 Team

The project is carried out by two students. Their confirmed identities and account-linked delivery responsibilities are recorded below; detailed contribution evidence is provided in Chapter 10.

Table 1.2. Team

Student	Group	GitHub account
Kozhabek Ayadil Nurlanuly	SE-2425	https://github.com/vbl0-0
Amantay Zhan Adiluly	SE-2419	https://github.com/igiiaw

2. Evidence of Relevance

2.1 Sources of evidence

Relevance at this stage is grounded in the scientific literature and system comparison in Chapters 3–4. User interviews were optional and were not conducted; local demand and user-comprehension claims therefore remain hypotheses.

2.2 Findings and assumptions

Table 2.1. Findings and assumptions

Claim	Available evidence	Limit
Skill extraction and normalization require explicit design decisions.	Scientific publications S01–S05.	Transfer to Kazakh/Russian and AITU profiles needs local evaluation.
Existing services combine vacancy matching and skill development.	Official system materials S11–S14 and the common-criteria comparison.	This does not prove unmet local demand or superiority of this prototype.
Explanations can expose covered, partial and missing skills.	Implemented calculation and automated functional checks.	User comprehension and usefulness have not been measured.
Employment readiness summarizes profile and vacancy evidence.	Documented configurable heuristic.	It is not a validated predictor of hiring success.

2.3 Further validation

Later validation may include anonymized discussions with graduates, career-center staff and recruiters. Record actual dates and consent, distinguish observations from interpretations, and link findings to requirement decisions.

3. Literature Review

3.1 Search scope

A targeted engineering search was performed on 19 October 2026 using ACL Anthology, arXiv and official product/technical documentation. Queries covered skill extraction from job postings, multilingual skill matching, sentence embeddings and job-title representations. Inclusion required verifiable metadata and a direct connection to extraction, retrieval or an engineering decision.

The review uses 17 sources: eight peer-reviewed scientific publications S01–S08, technical report S09 and eight official materials S10–S17. The scientific core is from 2021–2025; Sentence-BERT (2019) is retained as foundational work. The table connects the reviewed findings and limitations to engineering decisions for this project.

3.2 Literature comparison

Table 3.1. Literature comparison

Source and approach	Limitation for this project	Engineering implication
S01. Expert annotation of hard and soft skill mentions and domain-adapted extraction models.	English job postings do not establish performance on Russian/Kazakh texts or AITU profiles.	Evaluate extraction separately from ranking; retain reference spans and annotation rules.
S02. Survey of terminology, datasets and neural skill extraction methods.	Different datasets and task definitions prevent direct comparison of reported scores.	Define the annotation unit, skill terminology and normalization rules before evaluation.
S03. Multilingual domain-adaptive pre-training using the ESCO taxonomy.	Transfer to local occupations and languages requires validation; large-model adaptation needs resources.	Maintain canonical skills and aliases; consider ESCO alignment as a future extension.
S04. Few-shot LLM extraction explores complex skill mentions.	The study does not establish universal superiority over supervised methods; inference requires quality control and resources.	Keep external LLM calls optional and assess extraction against fixed labels.
S05. Multilingual vacancy–candidate matching through skill extraction, ESCO alignment and a knowledge graph.	The graph pipeline is more complex than a dictionary, and its benefit for AITU graduates remains unmeasured.	Compare normalization with semantic expansion and retain the origin of each match.
S06. Job-title representations incorporate information from descriptions beyond extracted skills.	Learning job-title representations is different from assessing an individual candidate.	Describe the current score as skill-requirement coverage, not a probability of being hired.
S07. Siamese sentence encoders enable efficient embedding-based text comparison.	Text similarity does not prove skill proficiency or satisfaction of all job conditions.	Use similarity only as supplementary evidence with a limited contribution.
S08. Contrastive learning produces sentence representations in supervised and unsupervised settings.	Semantic textual similarity benchmarks do not replace domain-specific matching evaluation.	Select an encoder using local ranking quality and latency as well as general benchmarks.
S09. A family of multilingual retrieval embeddings offers different model sizes.	Input conventions, model size and inference cost affect suitability.	Include E5 in a future comparison; it is not the model currently configured in the prototype.

3.3 Synthesis and research questions

S01–S04 motivate separating a skill mention, its canonical label and evidence of proficiency. The MVP therefore distinguishes an explicitly reported skill/level from inferred text evidence. Dictionary matching is reproducible but limited by catalog coverage and language variation. S03 and S05 motivate taxonomy alignment without establishing a need for a graph database at this scale.

S06 highlights information in vacancy descriptions beyond skill lists. The current score consequently measures only coverage of explicit or extracted skill requirements. S07–S09 justify evaluating embeddings against literal matching, but textual proximity alone does not establish competence. Semantic evidence is therefore capped at a partial contribution and its method is recorded.

RQ1: do alias normalization and optional semantic matching improve nDCG@5 over exact skill matching on a fixed profile/vacancy dataset while retaining interpretable explanations? RQ2: do users distinguish skill coverage from the probability of employment? The contribution is a testable engineering adaptation to AITU. Superiority over commercial systems is not asserted.

4. Existing System Comparison

4.1 Systems and criteria

Four relevant systems are compared on eight consistent criteria using official public documentation: LinkedIn [S11], Handshake [S12], hh.kz [S13] and Coursera Career Academy [S14]. No private product accounts were audited. “Not established” means that the cited sources do not establish the capability; it does not mean the capability is absent.

4.2 Comparison

Table 4.1. Comparison

Criterion	LinkedIn	Handshake	hh.kz	Career Academy
Primary purpose	Job discovery and matching	Early-career discovery and recommendations	Regional job search	Role-oriented skill development
User information	Profile and resume	Profile, interests and activity	Resume and key skills	Learning pathway; vacancy matching not established
Personalized vacancies	Documented	Documented	Exact mechanism not established by S13	Not established
Skills and requirements	Profile and job qualifications	S12 emphasizes interests and activity	Key skills	Skills associated with a career role
Match explanation	Documented	Per-requirement explanation not established	Not established	Role skills rather than individual vacancy fit
Development actions	Profile improvement and upskilling	Career recommendations; detail not established	Not established	Courses and practical projects
Published exact scoring formula	Not established	Not established	Not established	Not established
AITU-program alignment	Not established	Not established	Not established	Not established

4.3 Implications for AITU Career

Matching and skill-development recommendations already exist in competing systems. The rationale for this prototype is controllable AITU-specific adaptation, an accessible per-requirement calculation and reproducible evaluation on a fixed dataset. Optional future user research can test the local-demand hypothesis. Commercial systems are not quantitative baselines because their internal algorithms and evaluation data are unavailable.

5. Technology Comparison

5.1 Technology alternatives

The alternatives are assessed for a small educational prototype with a Python analysis module, relational data and no interaction history. Assessments are qualitative; no unperformed performance measurements are implied.

Table 5.1. Technology alternatives

Decision	Reasonable alternatives and trade-offs	Selected solution and rationale
Backend platform	FastAPI: validation/OpenAPI and direct Python NLP integration [S15]. Django REST: mature conventions and administration, with a larger framework surface. Node.js/NestJS: a shared frontend language, but Python NLP needs an additional integration boundary.	FastAPI supports one small API/analysis process and a modular code structure.
Data storage	PostgreSQL: relational constraints and transactions [S16]. SQLite: simple setup with a different concurrency/operation profile. MongoDB: flexible documents, with a different strategy for cross-record integrity.	PostgreSQL with SQLAlchemy and Alembic preserves linked profiles, catalog entries and vacancy requirements.
Matching approach	Exact matching: transparent but sensitive to wording. Dictionary/aliases/weights: explainable but requires catalog maintenance. Embedding hybrid: supports paraphrases [S07–S09] but adds calibration and inference cost.	Weighted normalized keyword matching is the default; embeddings remain optional and require separate evaluation.
Component communication	REST/JSON: straightforward CRUD and OpenAPI. GraphQL: flexible selections with an additional schema/cost-control layer. gRPC: typed binary communication but more integration work for a browser client.	REST/JSON under /api, with Vite/nginx proxying to a single browser origin.
Frontend	React/TypeScript: typed components already implemented. Vue/TypeScript: a comparable SPA option requiring component migration. Server templates: a smaller client stack but a different approach to interactive profile editing.	React/TypeScript with Vite follows the implemented journeys; no unmeasured speed advantage is claimed.

5.2 Architectural decisions

ADR-01: use a modular monolith. Authorization, API, analysis and persistence have separate code responsibilities but share one backend deployment. Revisit separation only if profiling demonstrates interference from inference. ADR-02: PostgreSQL is the source of truth; preserve algorithm version and input revisions with saved analyses. ADR-03: absence of the optional model must not break keyword analysis.

6. Requirements Specification

6.1 Priorities and traceability

MoSCoW priorities: Must denotes mandatory MVP behavior; Should denotes a planned improvement; Could denotes an optional extension. An implemented feature is not evidence that users need it. Requirement IDs connect the specification, components, tasks and verification.

6.2 Functional requirements

Table 6.1. Functional requirements

ID / priority	Requirement	Acceptance criterion	Verification
FR-01 / Must	Registration and sessions	A new email creates a graduate account; invalid passwords are rejected; logout invalidates the previous access token.	API authentication tests; browser login/logout
FR-02 / Must	Authorization boundaries	Other users' records are inaccessible; graduates cannot execute admin actions or grant themselves an admin role.	API ownership/role tests

ID / priority	Requirement	Acceptance criterion	Verification
FR-03 / Must	Profile and education	Program, graduation year and desired role are saved; an invalid graduation year is rejected.	API validation; graduate E2E
FR-04 / Must	Skills and proficiency	Catalog skills use levels 1–3; duplicates and unknown IDs are rejected.	API skill tests
FR-05 / Must	Portfolio	Projects, technologies, certificates and experience support CRUD; an end date before the start date is rejected.	API and portfolio E2E
FR-06 / Must	Vacancies and saved jobs	Search/filters return matching records; details expose requirements; repeated save/remove operations are handled consistently.	API vacancy tests; graduate E2E
FR-07 / Must	Deterministic matching	Identical keyword inputs/configuration produce the same score; four of five equal full matches yield 80%; no requirements yield null.	Unit matching tests
FR-08 / Must	Explanations	Each requirement belongs to matched/partial/missing; contribution, weight, method, version and gap-based actions are available.	Matching tests; detail/analysis inspection
FR-09 / Must	Stale analysis	Profile, vacancy or relevant context changes flag saved results as stale and offer recalculation.	API fingerprint/staleness tests; E2E
FR-10 / Must	Administration	Admins manage jobs/catalogs; deleting a referenced category is handled without data loss.	API constraints; admin E2E
FR-11 / Should	Semantic extension	No model triggers keyword fallback; a semantic match is labeled and contributes at most 0.5.	Controlled semantic/fallback tests; real-model evaluation separately
FR-12 / Must	Demonstration-data labels	Fictional vacancies are labeled; readiness reports its sample scope and components.	UI inspection; career summary

6.3 Non-functional requirements

Table 6.2. Non-functional requirements

ID / priority	Quality attribute	Acceptance criterion	Verification
NFR-01 / Must	Security	Argon2id passwords, signed JWTs, HttpOnly refresh cookies, hashed refresh secrets and rejection of untrusted origins.	API tests; configuration review
NFR-02 / Must	Integrity	Foreign keys, UNIQUE/CHECK constraints and migrations are enforced; repeat seeding creates no duplicates.	PostgreSQL tests; Alembic check
NFR-03 / Must	Reproducible startup	A supported clean environment can run setup, migrations, seed and health checks; native Windows and Compose instructions are separate.	Runbook; container startup not yet verified
NFR-04 / Must	Mobile usability	The core journey fits 390×844 without horizontal overflow; menu, inputs and actions remain available.	Playwright and screenshots
NFR-05 / Should	Accessibility	Keyboard focus is visible; normal text contrast is at least 4.5:1; errors have textual descriptions.	Targeted audit; comprehensive audit remains outstanding
NFR-06 / Should	Latency	Target: keyword matching p95 ≤500 ms on 30 demonstration vacancies after warm-up; no measurement is claimed.	30 warm-up and 200 measured requests on fixed hardware
NFR-07 / Must	Traceability	Saved results include algorithm version, mode, revisions and a context fingerprint.	Unit/API fingerprint tests
NFR-08 / Must	Confidentiality	Submission files exclude .env/tokens and identifiable interview data.	Package inventory and staged-diff review

6.4 Matching and readiness rules

Score = 100 × Σ(weight × contribution) / Σ(weight). For an explicit skill, contribution = min(1, user_level / required_level); for an absent skill it is zero. Inferred text and semantic evidence are capped at 0.5. Required/preferred skill weights default to 2/1 and may be set to other positive values. Scores across different vacancies do not imply equal job difficulty.

Readiness is an internal heuristic: market 60%, projects 15%, experience 10%, certificates 5% and education 10%, with configurable normalized weights. Market uses up to five highest-scoring target-role vacancies; relevant experience is capped at 180 non-overlapping days. These weights have not been empirically validated as predictors of employment.

6.5 AI functionality and implementation status

The implemented decision-support baseline is deterministic: it extracts known skill terms and aliases, combines profile evidence, scores vacancy requirements and derives gap-based actions. This is a knowledge-based matching pipeline, not a trained employment predictor or a generative assistant. The learned AI extension is a local sentence-embedding adapter; it is disabled in the verified configuration. The report therefore separates working rule-based functionality from available but unevaluated machine-learning functionality.

Table 6.3. AI functionality and implementation status

Capability	Actual mechanism and output	Evidence / current status
Skill extraction and normalization	Whole-term catalogue/alias matching in project descriptions, experience text and certificate names; canonical skill IDs and evidence sources are retained.	extract_skills() and competencies(); implemented and covered by tests. This is not an LLM or a general CV parser.
Proficiency-aware vacancy matching	Requirement weights and stored skill levels produce a coverage score and matched/partial/missing groups. Inferred evidence contributes at most 0.5; no requirements produce a null score.	score_requirements() and analyze(); implemented deterministic baseline, algorithm weighted-skills-1.0.
Career recommendations	Missing and partial requirements are sorted by importance and converted into skill-specific development actions. Readiness combines profile evidence with vacancy coverage.	analyze() and career_summary(); implemented rules, not learned generation or a probability of hiring.
Semantic skill matching	Optional SentenceTransformer encodes skill names; normalized dot-product similarity selects a candidate above the configured threshold. Semantic evidence remains partial.	semantic_candidate(); adapter exists, default threshold 0.85. Real model disabled/unavailable in the verified run; controlled tests and fallback only.
Learned ranking or employment prediction	A model trained on local hiring outcomes, validated success probabilities and an adaptive recommender are not implemented.	Future research possibilities; no effectiveness result is claimed.

6.6 Worked example

Illustrative calculation: Python level 3 of required 3 with weight 2 contributes 2; SQL level 1 of required 2 with weight 2 contributes 1; absent Git with weight 1 contributes 0. The score is 100 × (2 + 1 + 0) / (2 + 2 + 1) = 60%. Python is matched, SQL is partial and Git is missing.

Implementation reference: https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/backend/app/ai/matching.py. Matching correctness tests verify rules and boundaries. Only an independent relevance experiment can establish whether this approach ranks useful vacancies better than simpler alternatives.

7. Architecture and Diagrams

7.1 System context

Students/graduates and administrators use React, FastAPI and PostgreSQL. An explicit HeadHunter API adapter supports import, refresh, deduplication and source/availability tracking; live provider access was not available and no real import is claimed. University SSO and DU/Moodle are not integrated. Default keyword analysis needs no external AI service; the optional embedding model is disabled in the verified configuration.

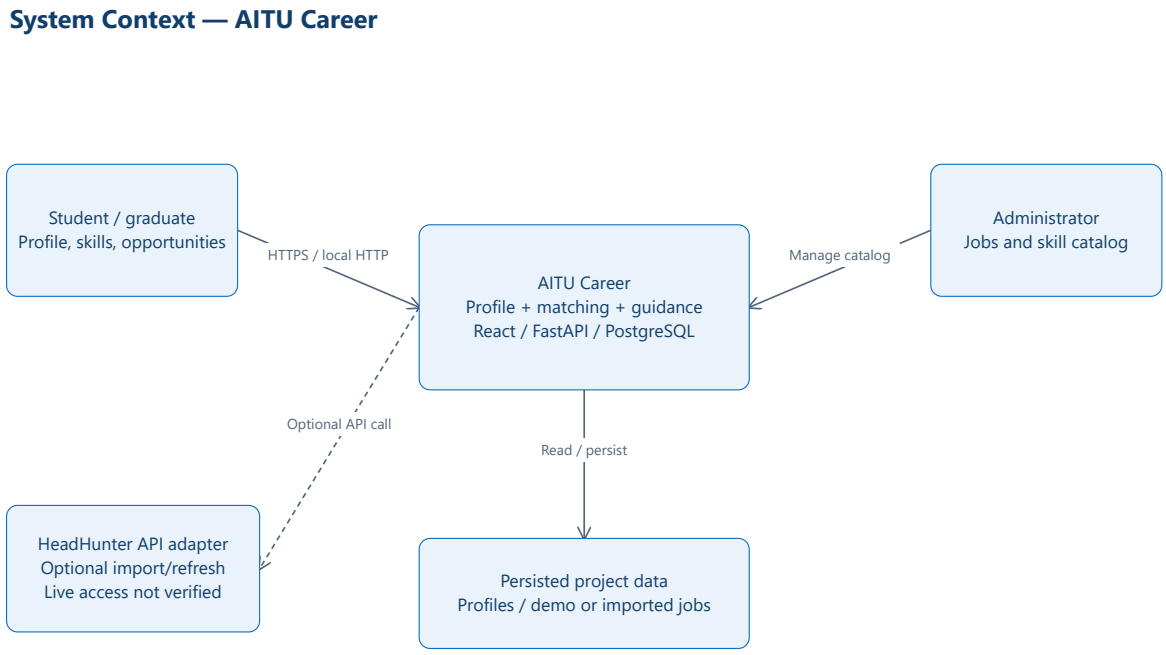


Figure 7.1. System context and responsibility boundaries.

7.2 Use cases

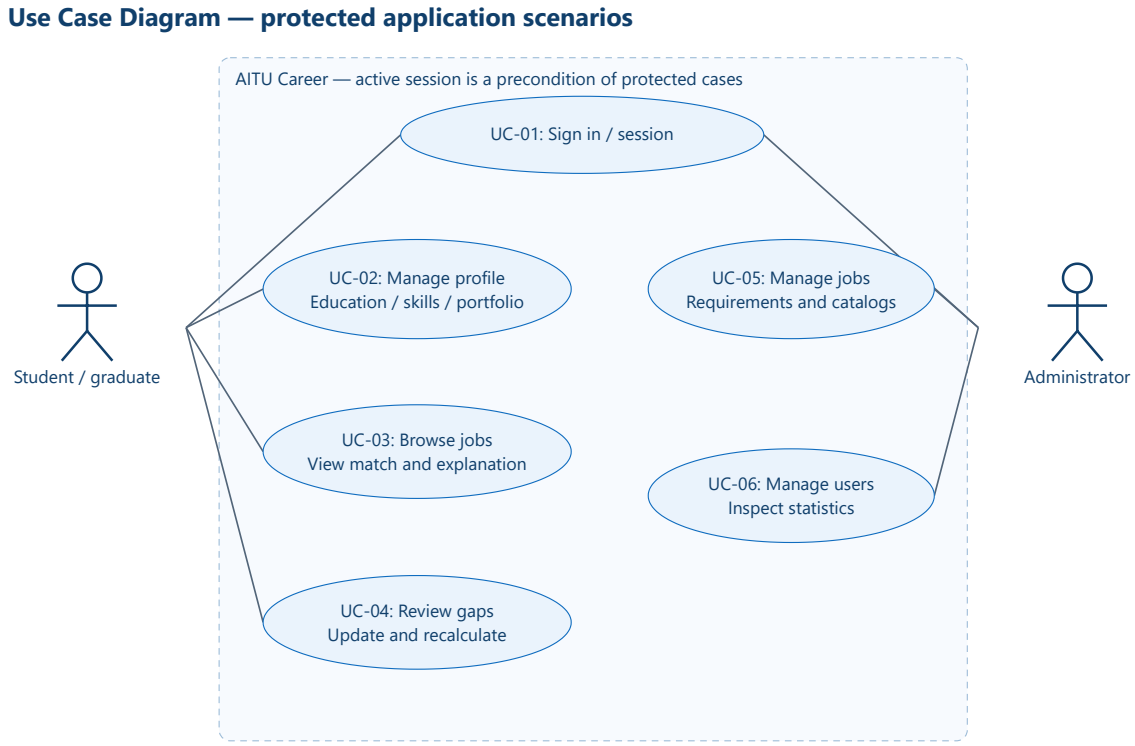


Figure 7.2. User roles and protected use cases.

7.3 Components

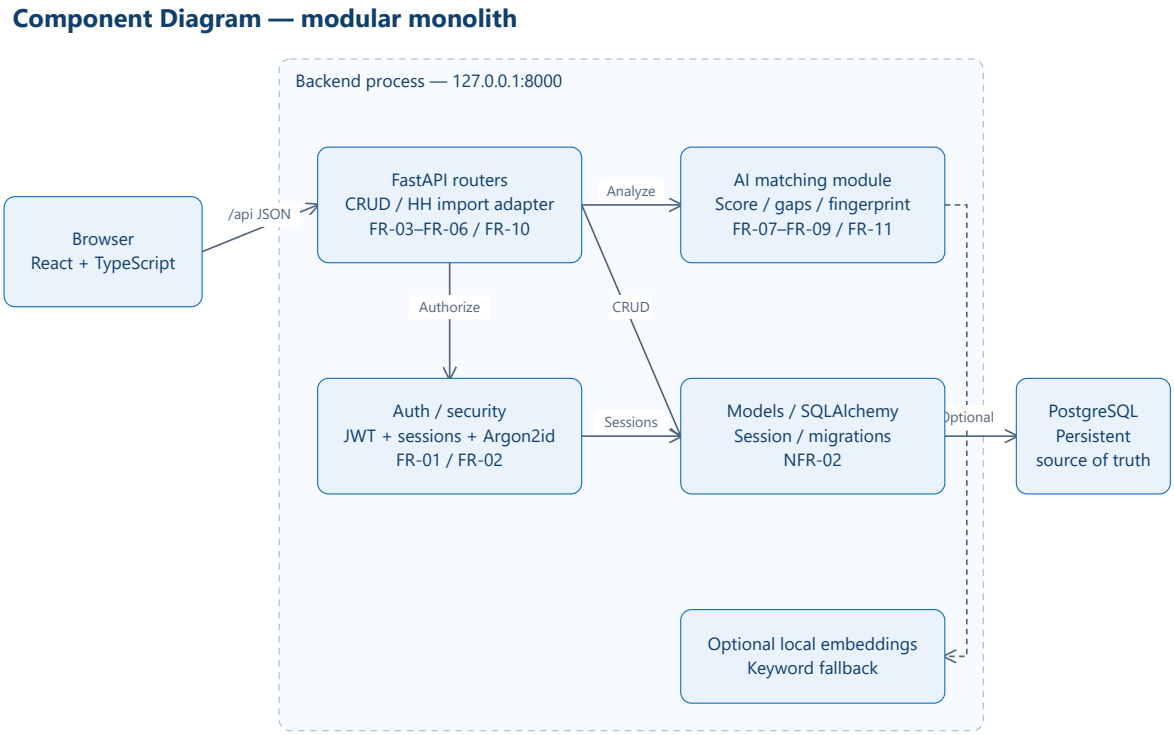


Figure 7.3. Draft component architecture aligned with the implementation.

Table 7.1. Components

Component	Responsibility	Related requirements
frontend/src/pages and components	Profile, vacancies, analysis and admin screens; routes and error states.	FR-03–FR-10; NFR-04/05
auth routes and core/security.py	Passwords, JWTs, sessions and permission checks.	FR-01/02; NFR-01
profile/jobs/admin routes	Input validation, CRUD, catalog and application journeys.	FR-03–FR-06; FR-10/12
ai/matching.py	Competencies, score, gaps, recommendations, fingerprint and optional semantics.	FR-07–FR-09/11; NFR-07
models/database/migrations	Relationships, constraints, persisted analyses and repeatable seed.	NFR-02/03

7.4 Data model and analysis flow

Data relationships: User 1:1 GraduateProfile; GraduateProfile 1:N Education, UserSkill, Project, Certificate, Experience, MatchResult and Recommendation. UserSkill/JobSkill link profiles/jobs to Skill; ProjectSkill links project technologies. SkillAlias stores normalization rules. CareerRole and categories are controlled dictionaries. User has AuthSession records; SavedJob links users to jobs. MatchResult preserves a calculated snapshot and its input/version context.

Analysis flow: authenticate and authorize → validate profile/job → normalize catalog terms → collect structured and textual evidence → obtain requirements → calculate weighted coverage → group matched/partial/missing requirements → derive development actions → persist the result with revisions → display it. Input changes do not silently replace the saved snapshot; they flag it as stale.

7.5 API and editable sources

Table 7.2. API and editable sources

API route / resource	Purpose
POST /api/auth/register, /login, /refresh, /logout	Account creation and session management.
/api/profile, /api/skills, /api/projects, /api/certificates, /api/experience	Profile and portfolio operations; supported methods depend on the resource.
GET /api/jobs; GET /api/jobs/{id}; POST /api/jobs/{id}/match	Vacancy discovery, details and matching.
GET /api/recommendations; GET /api/admin/statistics	Career overview and administrator statistics.
GET /api/health; /docs	Health check and generated API contract.

OpenAPI contract: https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/openapi.json. Local Swagger documentation: http://localhost:8000/docs. Editable diagram sources and exports: https://github.com/vbl0-0/AITU-Career/tree/prob-period-1/docs/pre-defense-1/diagrams.

8. UX and Prototype

8.1 Prototype access

Public screen walkthrough: https://vbl0-0.github.io/AITU-Career/. It presents actual captures of the local application with navigation between screens, not a hosted backend or live editing service. The complete application runs locally using the published development instructions. The interface supports Kazakh, Russian and English; the report is in English. Original screenshot and diagram colors are preserved.

8.2 User scenarios

Table 8.1. User scenarios

Journey	Steps	Expected outcome
UC-01: Start	Landing → registration/sign-in → dashboard.	An authenticated session and an accessible profile.
UC-02: Profile	Education program → desired role → skills → project.	Saved data and explicit confirmation.
UC-03: Match	Search → vacancy → score → detailed analysis.	Requirement groups and a traceable calculation.
UC-04: Improve	Add FastAPI → open an old analysis → recalculate.	A stale warning followed by a result based on current inputs.
UC-05: Administer	Admin sign-in → jobs → create/update.	Permission checks and validated skill requirements.

8.3 UX states and accessibility

UX states include loading, no records, validation errors, server failures, save confirmation and stale results. Focus indication, labels and accessible action names are implemented; a comprehensive accessibility audit remains outstanding. A user-evaluation task asks participants to find a relevant vacancy, identify two gaps and explain the score.

8.4 Screen examples

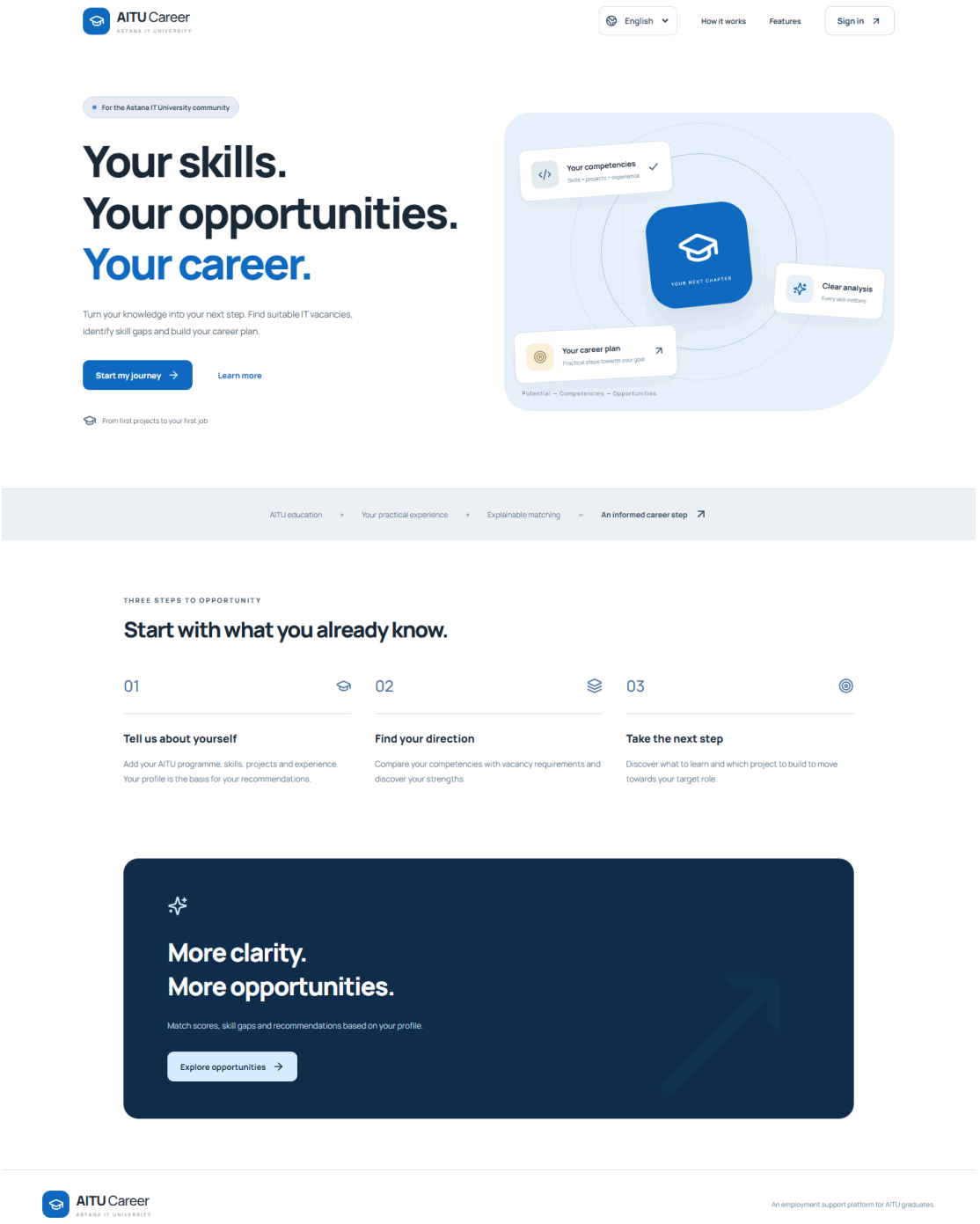


Figure 8.1. Entry to the graduate journey.

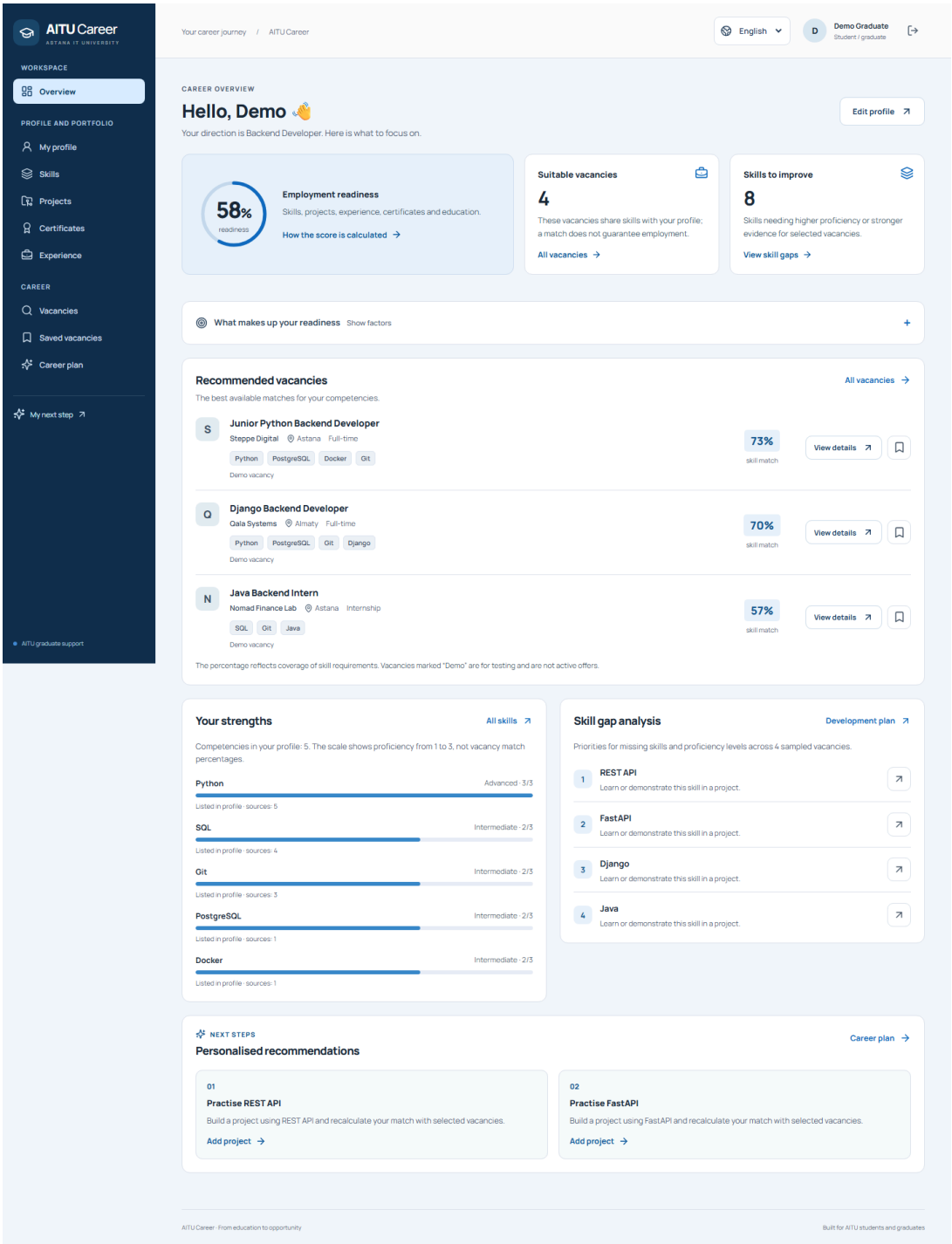


Figure 8.2. Demonstration graduate dashboard.

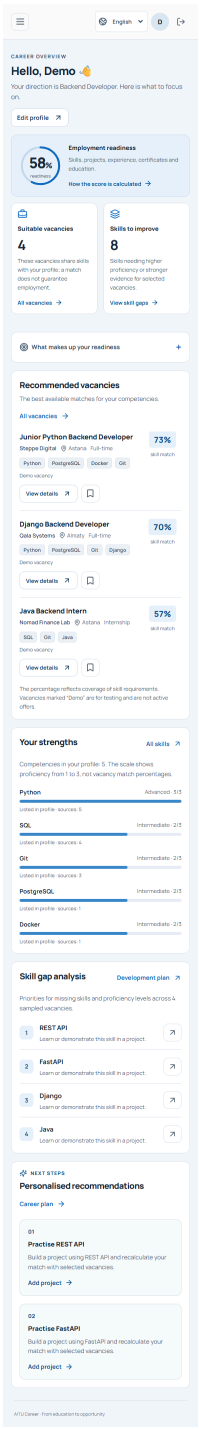


Figure 8.3. Mobile interface at 390×844.

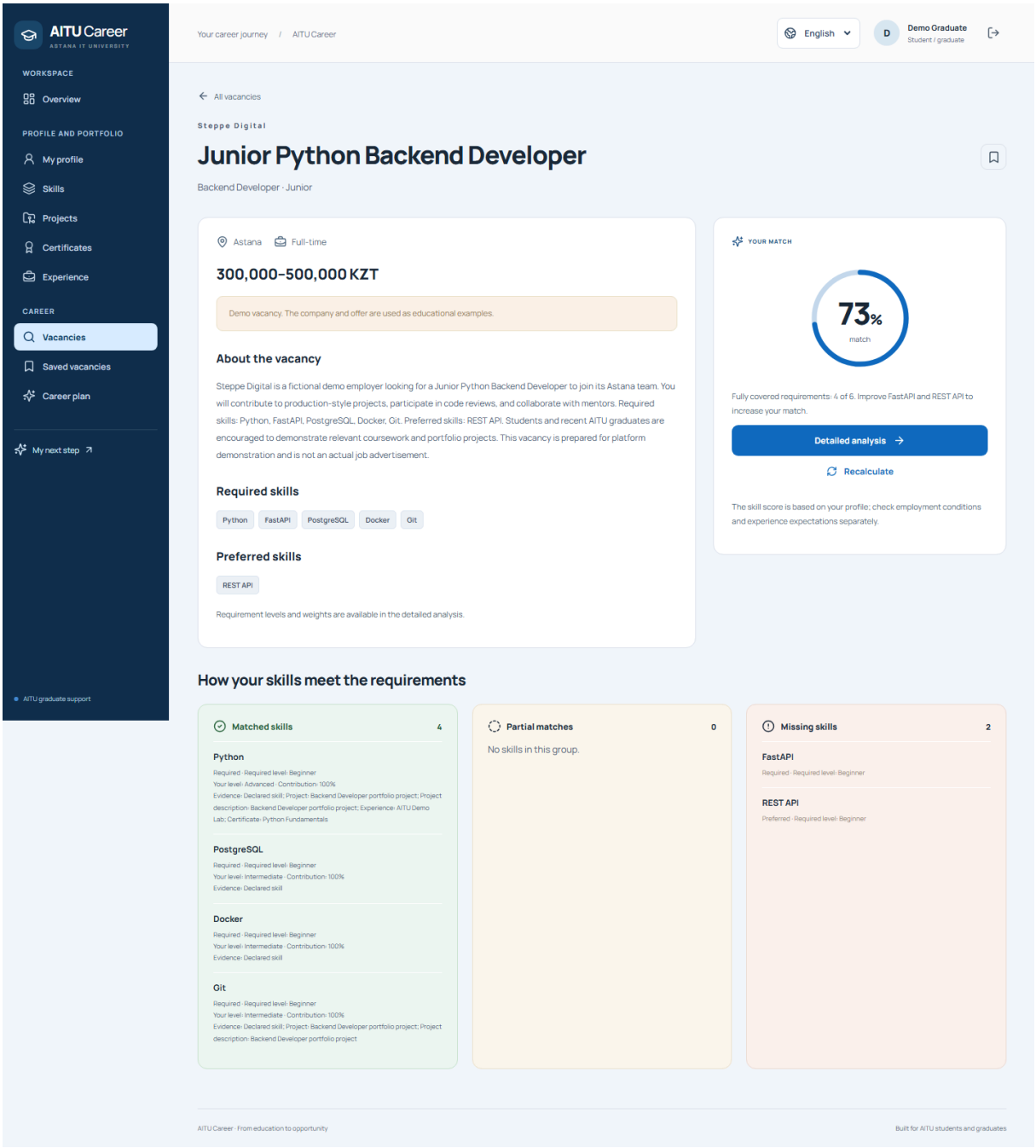


Figure 8.4. Vacancy requirements and explained matching.

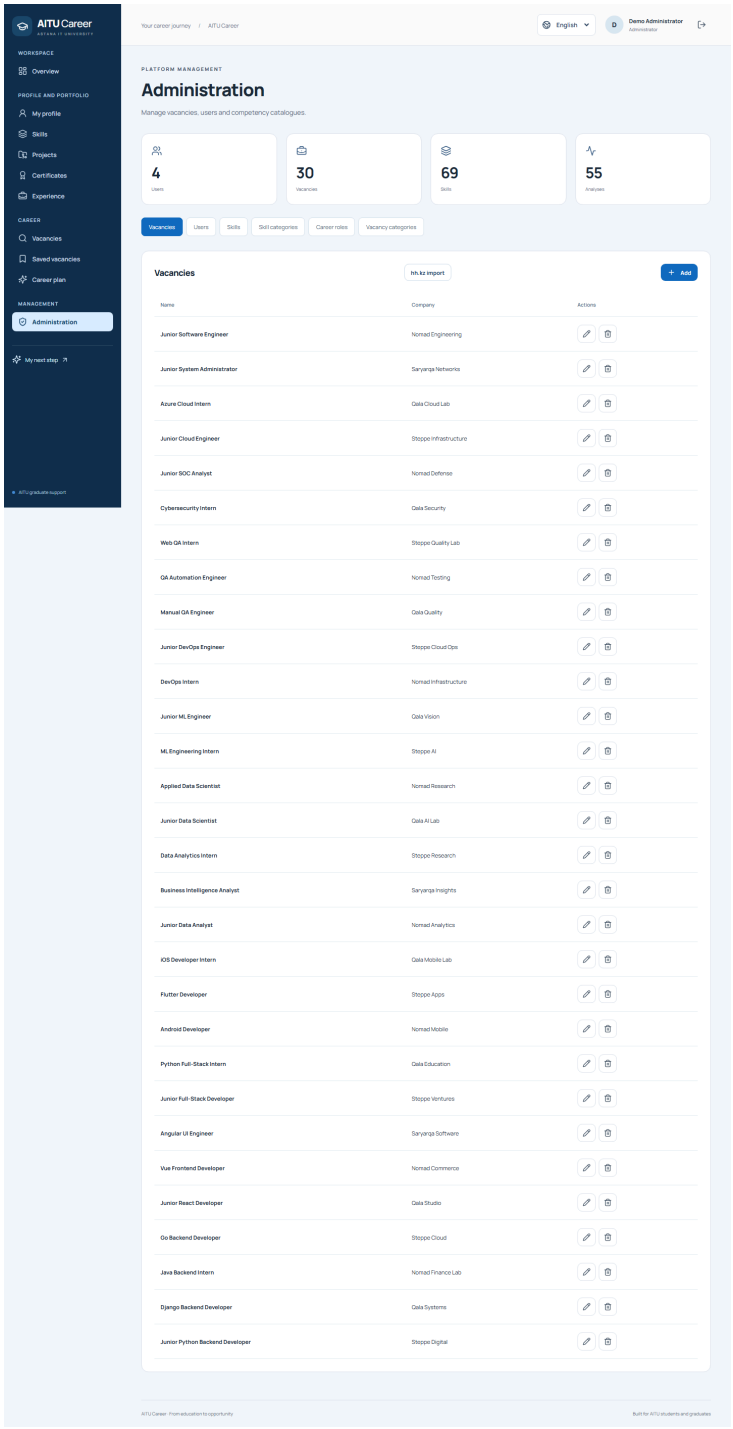


Figure 8.5. Administrative vacancy management.

9. Verification and Evaluation Plan

9.1 Verification results

Verification checks the software against the specified requirements. Evaluation compares matching quality and user interpretation under controlled conditions.

Table 9.1. Verification results

Verification on 24 October 2026	Observed result

Verification on 24 October 2026	Observed result
python -m pytest	passed: 60 checks passed. 35.06 seconds; one Starlette TestClient/httpx deprecation warning. The initial database connection timeout was resolved by starting the existing project cluster.
npm run test:e2e	passed: 20 checks passed. Chromium: graduate and admin journeys, portfolio CRUD, Dashboard states, role restrictions, responsive navigation, kk/ru/en and vacancy import/salary regressions. Some focused tests use API fixtures; the core journey and multilingual pages also use the real local API.
npm run check:i18n	passed: 459 checks passed. Translation keys, interpolation parity and static references in kk/ru/en.
npm run build	passed: TypeScript and production Vite build. Existing main-chunk warning: 501.13 kB before gzip.
python -m alembic upgrade head; python -m alembic check	passed: Migrations applied; no new upgrade operations detected.
node --test .github/scripts/project-sync.test.mjs	passed: 9 checks passed. Reference parsing, trusted PR filtering, transitions, partial merges, pagination and status-only/idempotent mutations.
Targeted UI review (eleven routes, desktop/mobile)	passed: 22 checks passed. Screenshots inspected; no horizontal document overflow or blocking page alert at 1440px/390px. Keyboard skip-link focus has a 3px outline. This is not comprehensive WCAG conformance.

Commands, environment details and observed verification results are retained in https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/validation.json.

Table 9.2. Verification results

Level	Verification focus	Artifact
Unit	Formula, levels, boundaries, aliases, partial evidence and semantic fallback/cap.	backend/tests/test_matching.py
API and database	Sessions, roles, ownership, CRUD, constraints and staleness.	backend/tests/test_api.py
Browser	Graduate journey, portfolio, administration and mobile viewport.	frontend/tests/career.spec.ts
Structure	Type checking, production build and model/migration consistency.	npm run build; alembic check
Quality evaluation	Independent labels and a fixed baseline comparison.	Protocol below; experiment not yet performed

9.2 Dataset and matching baselines

The following is a next-stage protocol, not a completed experiment. Freeze 20 anonymized or synthetic profiles and 50 legally usable vacancies, using at least five candidates per evaluated profile. Use ten profiles for development and ten held-out profiles for testing; store IDs, source permissions, split membership and data hashes. This produces 500 development and 500 held-out profile–vacancy pairs. Where development and test share vacancies, the stated claim concerns new profiles rather than unseen vacancies.

Table 9.3. Dataset and matching baselines

Comparison	Controlled change	Result status
B0 — exact overlap	Case-normalized canonical skill-name overlap, without aliases, proficiency levels or requirement weighting.	Not yet measured.

Comparison	Controlled change	Result status
B1 — current baseline	Current catalogue/aliases, evidence levels and requirement weights, with semantic inference disabled.	Implementation verified; ranking quality not yet measured.
B2 — semantic extension	B1 plus the optional local embedding adapter, with model revision and threshold fixed before the held-out run.	Real model not evaluated. If it cannot be loaded, record B2 as not run; do not substitute a fabricated score.

9.3 Independent labels and evaluation metrics

Two raters independent of the matching implementation should assign relevance grades 0 (irrelevant), 1 (partly relevant) and 2 (suitable), without seeing algorithm scores. Store both original ratings and weighted Cohen's kappa before adjudication. Rater identities and actual dates remain unfilled until the work occurs. Tune aliases and semantic thresholds only on development data; use the same eligible candidates for all methods and break score ties by vacancy ID.

Table 9.4. Independent labels and evaluation metrics

Measure	Calculation / procedure	Decision and retained artifact
Ranking relevance	Precision@5 = count of grade-2 vacancies in the first five / 5. nDCG@5 uses gains 2^grade − 1 and log2(rank + 1) discount, normalized by the ideal top-five ranking.	Macro-average per held-out profile. Profiles with zero ideal gain have undefined nDCG and are counted separately; report their number and candidate coverage. Preserve per-profile rankings and labels.
Extraction accuracy	Micro precision, recall and F1 against annotated canonical skill mentions; show separate Kazakh/Russian/English results and representative errors.	Keep span/skill reference annotations and the exact normalization rules. No extraction-accuracy figure is currently claimed.
Efficiency	Same CPU, RAM, data, process and cache conditions; 30 warm-up requests, then 200 measured requests. Record median and p95, model loading separately, and errors.	Keyword p95 target is at most 500 ms. Pre-register a separate semantic latency/memory budget before comparing B2. Preserve request timings and environment details.
Uncertainty and adoption	Use seed 42 and 1,000 paired profile-level bootstrap resamples for the nDCG difference; report a 95% interval and Precision@5 changes.	Adopt an extension only if held-out nDCG improves with an interval above zero, Precision@5 does not deteriorate, its registered resource budget is met and functional regressions are absent. Otherwise retain the simpler method and report the negative outcome.

9.4 Execution responsibilities and planned artifacts

Execution responsibilities are planned: Zhan coordinates the dataset, annotation records and error analysis; Ayadil prepares the reproducible B0/B1/B2 runner, model/configuration hashes and latency measurements. Both inspect the held-out results and review each other's experiment changes. Independent raters must be recruited. The tracker schedules label preparation in #10 and baseline/latency evaluation in #11 for the next stage.

Planned evaluation artifacts: a dataset/split manifest, independent labels and agreement record, a committed comparison runner, environment/model configuration, per-profile rankings, metrics/timing files and an interpretation of errors and negative results. The 60 backend and 20 browser tests establish implementation behavior, not AI effectiveness.

9.5 Validity threats

Validity threats include convenience sampling, self-reported skills, synthetic vacancies, language bias, tuning leakage, facilitator effects and a small profile sample. Mitigations are independent annotation, a held-out split, negative examples, fixed comparison conditions and retention of raw judgments and errors. Docker and the real embedding model were not exercised in this environment; controlled tests do not replace their evaluation.

10. Project Management and Risks

10.1 Team contributions

OpenAI Codex supported implementation, documentation and technical verification. The contribution table records dated deliveries under each account. Existing modules were imported on 23–24 October 2026; earlier individual development is not reconstructed. No independent peer review is recorded in the checked PRs #19–25. The next-stage review workflow is defined in Section 10.4.

Table 10.1. Team contributions

Participant / work area	Concrete delivered artifacts and traceability	Delivery record
Kozhabek Ayadil Nurlanuly / @vbl0-0	Backend API/models/migrations: 6c434f2; regression tests/container setup: 0b8c8bb; architecture/startup delivery: 26c8605. Related PRs #19, #21, #23 and issues #7/#9/#12/#14.	AI-assisted module delivery; linked commits and pull requests.
Amantay Zhan Adiluly / @igiiaw	Frontend/i18n/browser tests: 7717652; references/documentation: c12021f; report sources: 203f7c7; report/evidence exports: d308bb1 and adc5d46. Related PRs #20/#22/#24/#25 and issues #4/#5/#8/#16/#17.	AI-assisted module delivery; linked commits and pull requests.

Merged delivery records: https://github.com/vbl0-0/AITU-Career/pull/19 (repository automation); https://github.com/vbl0-0/AITU-Career/pull/20 (frontend); https://github.com/vbl0-0/AITU-Career/pull/21 (backend and regression tests); https://github.com/vbl0-0/AITU-Career/pull/22 (research and report sources). Further delivery PRs are recorded in the repository and linked issue comments. Original commits are preserved through merge commits.

10.2 Task allocation, deadlines and progress

Tracker: https://github.com/users/vbl0-0/projects/1. The 18 original tasks retain their source IDs, priorities, planned owners and original deadlines. GitHub Issues contain actual acceptance criteria, account assignments and completion evidence. Optional interview/feedback tasks are distinguished from completed work. Later-stage independent labels and matching-quality experiments are not marked complete.

Table 10.2. Task allocation, deadlines and progress

Task	Assignee	Original deadline	Status
#1: Record confirmed team details and project scope	@vbl0-0	2026-10-21	Completed
#2: Optional user discussions (not required for period 1)	@igiiaw	2026-10-22	Not planned
#3: Record requirements and confirmed scope clarifications	@vbl0-0	2026-10-22	Completed
#4: Review the literature and source analysis	@igiiaw	2026-10-22	Completed
#5: Review existing systems against common criteria	@igiiaw	2026-10-22	Completed

Task	Assignee	Original deadline	Status
#6: Review technology alternatives and ADRs	@vbl0-0	2026-10-22	Completed
#7: Review architectural diagrams	@vbl0-0	2026-10-22	Completed
#8: Verify the updated interface palette and UX	@igiiaw	2026-10-21	Completed
#9: Verify the application and preserve results	@vbl0-0	2026-10-22	Completed
#10: Prepare independent matching relevance labels	@igiiaw	2026-11-06	Open
#11: Evaluate matching baselines and latency	@vbl0-0	2026-11-13	Open
#12: Set up the shared repository and access	@vbl0-0	2026-10-22	Completed
#13: Set up the tracker and assign responsibilities	@igiiaw	2026-10-22	Completed
#14: Link code changes and reviews to real tasks	@vbl0-0	2026-10-23	Completed
#15: Record received supervisor feedback and response	@igiiaw	2026-10-23	Completed
#16: Finalize report and record any optional feedback	@igiiaw	2026-10-23	Completed
#17: Compile actual board and task screenshots into PDF	@igiiaw	2026-10-23	Completed
#18: Verify access and submit the team materials	@vbl0-0	2026-10-24	Open

The deadlines above are the original recorded planning dates. They are retained alongside the actual issue states; completion evidence and timestamps are available on the linked GitHub issues.

Tracker snapshot: 14 completed, 1 optional activities closed as not planned, and 3 open. GitHub's built-in closed-item workflow places both closure types in Done; the issue state reason distinguishes cancelled scope from performed work. Snapshot: https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/github-current-issues.json.

10.3 Recorded development history

The original Git metadata records repository setup, automation and frontend import on 23 October 2026, followed by backend, report-source, architecture and submission deliveries on 24 October 2026 (UTC+5 author timestamps). The pre-feedback snapshot contains 11 non-merge commits, preserved with their original author dates and subjects: https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/development-history.json.

10.4 Git workflow and reciprocal peer review

The supervisor’s general guidance calls for a traceable development process and mutual review, rather than publication of completed files alone. The existing delivery history and task records are retained. The following process is planned for the next substantive changes; independent approvals and a protected-main review gate have not yet been introduced.

- Create or refine a task with one responsible student, an expected result, acceptance criteria, deadline and current status.
- Create a task branch from the current main branch and make small commits that identify the actual task and the change performed.
- Open a pull request explaining the change, the task link, relevant tests and any AI assistance. Update the README and related documentation when behavior or setup changes.
- The other student personally reviews the diff and runs the relevant checks: Zhan reviews Ayadil’s backend changes; Ayadil reviews Zhan’s frontend and report changes.

- Record substantive comments, link corrective commits and obtain a recheck before approval. Preserve the actual review discussion and the participant’s implementation, testing and review actions.
- Merge only after acceptance checks and the required review are satisfied. Add the resulting commit or pull-request link to the task and then update its status.
- Before each submission, verify all shared links and the complete README, and confirm that the report and artifacts match the submitted tag. Preserve versions after they have been submitted.

The proposed repository gate is one independent approval with the relevant checks passing before merge. Its activation and actual human review records remain next-stage work. The checked review-status snapshot is retained in https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/peer-review-status-2026-10-24.json.

10.5 Risks

Table 10.3. Risks

Risk or remaining limit	Response
Static prototype versus the complete application	The public viewer is clearly labelled; use the development guide to run authenticated CRUD and matching.
Demonstration vacancies mistaken for live jobs	The seed dataset is fictional and labelled. No live HeadHunter import or current market representativeness is claimed.
Unmeasured model or employment outcomes	Keep the evaluation protocol; do not infer efficacy from passing implementation tests.
Confusing account history with individual authorship	Preserve real dates and links; state the use of AI assistance and distinguish delivery from historical authorship.
Documents differ from the submitted version	Use the exact prob-period-1 tag and the matching PDF exports; do not move the submitted tag.
Different evidence requirements are confused	Keep interviews optional for this stage, record the received supervisor feedback, and introduce reciprocal peer review for subsequent development.

11. Supervisor Feedback

11.1 Reviews received

Written supervisor feedback dated 24 October 2026 was supplied to the team. The messages below contain the assessment and requested improvements. They refer to the pre-defence materials without specifying an exact reviewed commit.

24 October 2026, 15:28 — original English message:

“The team has made good progress in project analysis, system design, documentation, and initial implementation. The report, GitHub repository, task tracker, and prototype are well organized and generally meet the objectives of the current stage. For the next stage, the team should focus on further system testing, evaluating the matching approach, and improving development and code review practices.”

24 October 2026, 15:29 — English translation of the Russian message:

“Overall, the team has worked well and the report is fairly complete. However, several aspects need improvement: make the AI functionality and the method for evaluating its effectiveness clearer. It is also important to show each participant's actual contribution, the development history, and to add peer

review in GitHub.”

The original text and translation are retained in https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/supervisor-feedback-2026-10-24.json.

11.2 Comments and decisions

Table 11.1. Comments and decisions

Supervisor comment	Change in this report	Implementation / next-stage status
Clarify the AI functionality.	Section 6 now distinguishes deterministic extraction/matching/recommendation rules, the optional embedding adapter and functionality that does not exist; includes a worked calculation and code reference.	Documentation updated. No new trained model or generative service is claimed.
Explain how matching effectiveness will be evaluated.	Section 9 specifies B0/B1/B2, data splits, blind independent labels, metrics, latency conditions, adoption criteria and retained artifacts.	Protocol clarified. Independent quality and latency experiments remain planned in #10/#11.
Show each participant's actual contribution and development history.	Section 10 lists concrete account-linked artifacts, commit IDs, PRs, real author dates and limits of historical authorship attribution.	Published evidence documented. Subsequent contribution logs must record actual personal implementation, testing and review actions.
Add peer review and improve development practices.	Section 10 defines cross-review responsibilities, task-focused PRs, review evidence, follow-up fixes and a proposed independent-approval gate.	Not yet performed/configured. No GitHub peer approval is fabricated.
Continue system testing.	Section 9 preserves actual 60-backend/20-browser verification separately from independent model evaluation and identifies untested areas.	Current verification recorded; further regression coverage and quality experiments belong to the next stage.
General guidance: demonstrate real development history, individual task ownership, deadlines/statuses and reciprocal review; keep README and shared links current.	Section 10 presents the actual task register, original commit evidence and reciprocal review workflow; the submission summary maps chapters to repository files.	Existing delivery records documented. Independent human review remains planned; README and link checks form part of each release check.

11.3 Follow-up

The present revision clarifies the AI functionality, evaluation protocol and delivery evidence. Independent matching evaluation and reciprocal GitHub peer review remain prospective actions in Chapters 9–10. Further comments will be recorded with the affected artifact, the resulting task and the corrective commit or pull request.

The additional general instruction is retained in https://github.com/vbl0-0/AITU-Career/blob/prob-period-1/docs/pre-defense-1/evidence/general-development-guidance.json; its original message timestamp was not supplied.

Appendix A. References

[S01] Zhang M., Jensen K., Sonniks S., Plank B. (2022). SkillSpan: Hard and Soft Skill Extraction from English Job Postings. NAACL, 4962–4984. Scientific publication. https://aclanthology.org/2022.naacl-main.366/ Accessed 19 October 2026.

[S02] Senger E., Zhang M., van der Goot R., Plank B. (2024). Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings. NLP4HR, 1–15. Scientific publication. https://aclanthology.org/2024.nlp4hr-1.1/ Accessed 19 October 2026.

[S03] Zhang M., van der Goot R., Plank B. (2023). ESCOXLM-R: Multilingual Taxonomy-driven Pre-training for the Job Market Domain. ACL, 11871–11890. Scientific publication. https://aclanthology.org/2023.acl-long.662/ Accessed 19 October 2026.

[S04] Nguyen K., Zhang M., Montariol S., Bosselut A. (2024). Rethinking Skill Extraction in the Job Market Domain using Large Language Models. NLP4HR, 27–42. Scientific publication. https://aclanthology.org/2024.nlp4hr-1.3/ Accessed 19 October 2026.

[S05] Kavas H., Serra-Vidal M., Wanner L. (2025). Multilingual Skill Extraction for Job Vacancy–Job Seeker Matching in Knowledge Graphs. GenAIK, 146–155. Scientific publication. https://aclanthology.org/2025.genaik-1.15/ Accessed 19 October 2026.

[S06] Laosaengpha N., Tativannarat T., Piansaddhayanon C., Rutherford A., Chuangsuwanich E. (2024). Learning Job Title Representation from Job Description Aggregation Network. Findings of ACL, 1319–1329. Scientific publication. https://aclanthology.org/2024.findings-acl.77/ Accessed 19 October 2026.

[S07] Reimers N., Gurevych I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 3982–3992. Scientific publication; foundational work. https://aclanthology.org/D19-1410/ Accessed 19 October 2026.

[S08] Gao T., Yao X., Chen D. (2021). SimCSE: Simple Contrastive Learning of Sentence Embeddings. EMNLP, 6894–6910. Scientific publication. https://aclanthology.org/2021.emnlp-main.552/ Accessed 19 October 2026.

[S09] Wang L., Yang N., Huang X., Yang L., Majumder R., Wei F. (2024). Multilingual E5 Text Embeddings: A Technical Report. arXiv:2402.05672. Technical report / preprint. https://arxiv.org/abs/2402.05672 Accessed 19 October 2026.

[S10] European Commission. What is ESCO? Official classification. https://esco.ec.europa.eu/en/about-esco/what-esco Accessed 19 October 2026.

[S11] LinkedIn Help. Find how you match up with jobs on LinkedIn. Official product documentation. https://www.linkedin.com/help/linkedin/answer/a7120158 Accessed 19 October 2026.

[S12] Handshake Help Center. Jobs Digests and Recommendations. Official product documentation. https://support.joinhandshake.com/hc/en-us/articles/360000684947-Jobs-Digests-and-Recommendations Accessed 19 October 2026.

[S13] hh.kz. Digitizing candidate skills: guidance on key skills in vacancies. Official product documentation. https://hh.kz/article/25442 Accessed 19 October 2026.

[S14] Coursera. Introducing Career Academy. Official product documentation. https://blog.coursera.org/introducing-career-academy/ Accessed 19 October 2026.

[S15] FastAPI. Features. Official technical documentation. https://fastapi.tiangolo.com/features/ Accessed 19 October 2026.

[S16] PostgreSQL. About. Official technical documentation. https://www.postgresql.org/about/ Accessed 19 October 2026.

[S17] Sentence Transformers. Semantic Textual Similarity. Official technical documentation. https://www.sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html Accessed 19 October 2026.

Appendix B. Submission Checklist and Evidence Status

Submission form: https://forms.cloud.microsoft/r/Yy6AXT5pe3. The provided screenshots specify a repository URL, the exact prob-period-1 version URL, individual contributions, current status, one board/task PDF (maximum 10 MB) and one consolidated report PDF (maximum 100 MB). This report contains the eleven main sections in order and a table of contents.

Table 13.1. Overview

Requirement	Prepared artifact	Scope or remaining limit
Repository and version	https://github.com/vbl0-0/AITU-Career; https://github.com/vbl0-0/AITU-Career/tree/prob-period-1	The tag identifies the submitted package.
Identity and contributions	Confirmed names, groups, supervisor and account-linked delivery records.	AI assistance and current-delivery limits are disclosed.
Research and comparisons	17 sources including eight scientific publications; four systems and eight common criteria; five technology decisions.	Local user demand and real matching quality are not empirically established.
Requirements and architecture	Testable FR/NFR criteria and editable diagram sources/exports.	https://github.com/vbl0-0/AITU-Career/tree/prob-period-1/docs/pre-defense-1/diagrams
Prototype	https://vbl0-0.github.io/AITU-Career/	A public screen walkthrough; the complete API-backed application runs locally.
Verification	Current backend, browser, build, translation and migration results.	Docker runtime and real embedding-model evaluation were not performed.
Interviews and supervisor feedback	No interviews; actual supervisor messages of 24 October and the response matrix are recorded in Section 11.	Evaluation and human peer review remain next-stage work.
Task evidence PDF	tracker-evidence.pdf with genuine board and task captures.	Tasks show actual progress; later-stage work can remain open.
Unified editable report	report.docx, report.html and matching report.pdf.	Use the final named PDF copy for the form.
Form submission	Prepared answers and both upload files.	The representative must verify the final form page and retain the actual confirmation.

Report upload filename: SE-2425_Kozhabek_ProbPeriod1.pdf. Deadline: 24 October 2026 before 18:00 UTC+5. Known fields are completed; any genuinely unknown personal information stays blank. The package does not assert that the form has already been submitted.